

소프트웨어공학

소프트웨어 공학의 발전적 추세 1

1 소프트웨어 재사용 기법

2 소프트웨어 재공학

3 소프트웨어 역공학

01

소프트웨어 재사용 기법

- 소프트웨어공학 3R의 정의
 - 저장소를 기반으로 역공학, 재공학, 재사용을 통해 SW 생산성을 극대화하는 기법
 - Reverse Engineering
 - Re-Engineering
 - Reuse

01

소프트웨어 재사용 기법

- 소프트웨어공학 3R의 목표
 - 시스템의 이해, 변경, 테스트 용이
 - SW 유지보수 용이성 향상으로 비용 절감
 - 현 시스템의 컴포넌트 재사용을 통해 SW 위기 극복

구성요소	설명
역공학	- Reverse Engineering - 구현된 것을 분석하여 처음의 문서나 설계기법 등의 자료를 얻어 내는 일
재공학	- Re-Engineering - 역공학으로 재구조화된 SW를 기반으로 다시 추상개념을 현실화하는 작업
재사용	- Reuse - 재공학을 통해 구현된 SW를 재사용하는 것

01

소프트웨어 재사용 기법

- 소프트웨어공학 3R의 목표

소프트웨어 재사용 (software reuse)

이미 개발되어 인정받은 소프트웨어의 전체 혹은 일부분을 다른 소프트웨어 개발이나 유지에 사용하는 것이다.

- 소프트웨어 개발의 품질과 생산성을 높이기 위한 방법으로, 기존에 개발된 소프트웨어 와 경험, 지식 등을 새로운 소프트웨어에 적용한다.
- 클래스, 객체 등의 소프트웨어 요소는 소프트웨어 재사용성을 크게 향상시켰다.
- 소프트웨어 부품(모듈)의 크기가 작고 일반적일수록 재사용률이 높다.

소프트웨어 재사용 기법

- 소프트웨어 재사용의 장점
 - 소프트웨어를 재사용함으로써 주어지는 장점은 다음과 같다.
 - 개발 시간과 비용을 단축시킨다.
 - 소프트웨어 품질을 향상시킨다.
 - 소프트웨어 개발의 생산성을 향상시킨다.
 - 프로젝트 실패의 위험을 감소시킨다.
 - 시스템 구축 방법에 대한 지식을 공유하게 된다.
 - 시스템 명세, 설계, 코드 등 문서를 공유하게 된다.

소프트웨어 재사용 기법

- 재사용 도입의 문제점

- 어떤 것을 재사용할 것인지 선정해야 한다.
- 시스템에 공통적으로 사용되는 요소들을 발견해야 한다.
- 프로그램의 표준화가 부족하다.
- 새로운 개발 방법론을 도입하기 어렵다.
- 재사용을 위한 관리 및 지원이 부족하다.
- 기존 소프트웨어에 재사용 소프트웨어를 추가하기 어렵다.
- 프로그램 언어가 종속적이다.
- 소프트웨어 요소의 내부뿐만 아니라 인터페이스 요구사항의 이해가 필요하다.
- 라이브러리 안에 포함시킬 재사용 요소의 명확한 결정 기준이 없다.

소프트웨어 재사용 기법

- 재사용이 가능한 소프트웨어 요소
 - 전체 프로그램, 부분 코드, 응용 분야에 관한 지식, 논리적인 데이터 모형, 프로세스 구조, 시험 계획, 설계에 관한 결정, 시스템 구조에 관한 지식 등
 - 프로그램 코드 : 전체 코드와 부분 코드
 - 설계 명세 : 논리적 데이터 모형, 프로세스 구조, 응용 모형
 - 계획 : 프로젝트 관리 계획, 시험 계획
 - 문서화와 문서 작성에 사용된 모든 정보들
 - 전문적인 기술과 경험 : 생명주기 모형 재사용, 품질 보증 기법, 응용분야 지식

소프트웨어 재사용 기법

- 재사용 라이브러리와 재사용 요소
 - 재사용 라이브러리의 속성
 - 편리한 접근, 탐색, 버전, 제어 그리고 보안을 제공하는 DBMS
 - 재사용 요소들의 생성, 편집 등을 허용하는 연산
 - 표준화된 요소 표현 형식
 - 확장성
 - 재사용 요소의 생성
 - 널리 재사용될 수 있는 요소의 확인
 - 요소의 확인과 재사용 재고를 위한 제공학
 - 재사용 요소의 저장과 분류
 - 재사용이 가능한 요소를 표준형식으로 표현
 - 재사용 요소의 생성과 정제

01

소프트웨어 재사용 기법

- 재사용 방법

- 소프트웨어 재사용 방법에는 합성 중심 방법과 생성 중심 방법이 있다.

합성 중심 (composition-based)	전자 칩과 같은 소프트웨어 부품, 즉 블록(모듈)을 만들어서 끼워 맞추어 소프트웨어를 완성시키는 방법으로, 블록 구성 방법이라고도 한다.
생성 중심 (generation-based)	추상화 형태로 쓰인 명세를 구체화하여 프로그램을 만드는 방법으로, 패턴 구성 방법이라고도 한다.

소프트웨어 재공학 (software reengineering)

새로운 요구에 맞도록 기존 시스템을 이용하여 보다 나은 시스템을 구축하고, 새로운 기능을 추가하여 소프트웨어 성능을 향상시키는 것이다.

- 유지보수 비용이 소프트웨어 개발 비용의 대부분을 차지하는 문제를 염두에 두어 기존 소프트웨어의 데이터와 기능들의 개조 및 개선을 통해 유지보수성과 품질을 향상시키려는 기술이다.
- 유지보수 생산성 향상을 통해 소프트웨어 위기를 해결하는 방법이다.
- 기존 소프트웨어의 기능을 개조하거나 개선하므로, 예방(preventive) 유지보수 측면에서 소프트웨어 위기를 해결하는 방법이라고 할 수 있다.

소프트웨어 재공학 (software reengineering)

새로운 요구에 맞도록 기존 시스템을 이용하여 보다 나은 시스템을 구축하고, 새로운 기능을 추가하여 소프트웨어 성능을 향상시키는 것이다.

- 소프트웨어 재공학도 자동화된 도구를 사용하여 소프트웨어를 분석하고 수정하는 과정을 포함한다.
- 소프트웨어 수명이 연장되고, 소프트웨어 기술이 향상된다.
- 소프트웨어에서 발생할 수 있는 오류가 줄어들고, 비용이 절감된다.

- 재공학의 등장배경
 - 기존의 소프트웨어가 노후되어 새로운 소프트웨어로 대체해야 할 경우
현재 시스템보다 훨씬 더 좋은 시스템을 만들 수 있다는 보장이 없다.
 - 현재 소프트웨어 품질이 더 좋은 소프트웨어로 교체된다고 해도 사용상의
문제점이 없다고 장담할 수 없다.
 - 새로운 소프트웨어 개발에도 기존 시스템과의 호환성이 100% 이루어질 수도
없을 뿐만 아니라, 사용자의 교육에도 많은 영향을 줄 수 있다.

- 재공학의 목표

- 소프트웨어 재공학은 유지보수성 및 기술 향상, 유지보수의 생산성 향상, 소프트웨어 수명연장, 소프트웨어 요소들을 추출하여 정보저장소에 저장하는 것을 주된 목적

복잡한 시스템을 다루는 방법 구현	시스템이 복잡해질수록 시스템을 다루는 방법이 필요하여 이를 위해 자동화 도구 등을 사용할 수 있다.
다른 뷰의 생성	기존 시스템 개발에 대한 관점 외에 다른 방향의 관점을 생성한다.
잃어버린 정보의 복구 및 제거	시스템이 계속적인 개발을 거치면서 잃어 버린 정보를 복구하거나 필요없는 정보를 제거한다.
부작용의 발견	의도되지 않았던 내용이 구현될 경우 이를 발견한다.
고수준의 추상	추상화된 어려운 내용을 여러 형태로 추출하여 이해에 도움을 준다.
재사용 용이	재사용이 가능한 모듈을 추출하여 재사용이 용이하도록 한다.

02

소프트웨어 재공학

- 소프트웨어 재공학의 주요 활동
 - 분석
 - 개조(재구성)
 - 역공학
 - 이식

- 소프트웨어 재공학의 분석 활동

분석	기존 소프트웨어의 명세서를 확인하여 소프트웨어의 동작을 이해하고, 재공학 대상을 선정하는 것이다.
- 최선의 대안 규명하기 위함 - 기존 시스템의 원시코드, 문서화를 조사 분석하는 활동 - 정적 분석과 동적분석으로 구분	

정적분석

코드의 라인수
 분기문의 수
 수행 또는 사용되지 않는 라인수
 비구조적 코드의 비율
 주석이 없는 코드의 비율
 복잡도와 품질 척도

동적분석

프로그램 수행경로 수
 기존 코드의 분석 및 이해에 소요되는 시간
 데이터 개체나 항목을 검색 수정하는 라인들
 하나의 변수값을 변경시의 프로그램 반응

- 소프트웨어 재공학의 개조 활동

개조(재구성)

- 개조(restructuring)는 상대적으로 같은 추상적 수준에서 하나의 표현을 다른 표현 형태로 바꾸는 것이다.
- 기존 소프트웨어의 구조를 향상시키기 위하여 코드를 재구성하는 것으로 소프트웨어의 기능과 외적인 동작은 바뀌지 않는다.

- 논리적 구조를 표준화하여 유지보수성과 생산성을 향상하기 위한 활용
- 프로그램 논리 개조와 데이터 개조로 구분

소프트웨어 재공학

- 소프트웨어 재공학의 개조 활동
 - 논리 개조
 - 기본기능
 - ❖ 프로그램 논리의 재정렬, 원시 코드의 재형식, 언어 사용의 표준화
 - 잘 구조화된 프로그램의 목적
 - ❖ 판독성과 신뢰성 향상
 - ❖ 복잡도의 최소화
 - ❖ 유지보수의 간소화
 - ❖ 생산성 증대
 - ❖ 프로그래밍에 대한 훈련 제공

- 소프트웨어 재공학의 개조 활동
 - 데이터 개조
 - 장점
 - ❖ 기존 시스템의 이해성 개선
 - ❖ 유지보수 비용 감소
 - ❖ 유지보수자의 생산성 증가
 - ❖ 데이터 문서화 개선
 - ❖ 데이터 사전과 모델링의 생성 지원
 - ❖ 데이터 문서화 개선
 - ❖ 기술 이전을 위한 시스템 배치
 - ❖ 중복되거나 비구조적인 데이터 요소 제거

- 소프트웨어 재공학의 개조 활동
 - 데이터 개조
 - 데이터 개조의 기능
 - ❖ 데이터 레코드 분석과 갱신
 - ❖ 데이터 요소 검색과 갱신
 - ❖ 자료 사전 적재
 - ❖ 연산 환경 분석

02

소프트웨어 재공학

- 소프트웨어 재공학의 주요 활동

- 이식

이식
(migration)

기존 소프트웨어를 다른 운영체제나 하드웨어 환경에서 사용할 수 있도록 변환 하는 작업이다.

03

소프트웨어 역공학

- 역공학

역공학
(reverse engineering)

기존 소프트웨어를 분석하여 소프트웨어 개발 과정과 데이터 처리 과정을 설명하는 분석 및 설계 정보를 재발견하거나 다시 만들어 내는 작업이다.

- 정공학(일반적인 개발단계)과는 반대방향으로 기존 코드를 복구하는 방법이다.
- 대상 소프트웨어가 있어야 하며 이로부터 작업이 시작된다.
- 기존 소프트웨어의 구성요소와 그 관계를 파악하여 설계도를 추출하거나, 구현과는 독립적인 추상화된 표현을 만든다.

03

소프트웨어 역공학

- 역공학

코드 역공학	코드 → 흐름도 → 자료 구조도 → 자료 흐름도 순으로 재생한다.
데이터 역공학	코드 → 자료 사전 → 개체 관계도 순으로 재생한다.

- 역공학의 가장 간단하고 오래된 형태는 재문서화(redocumentation)이다.